

GDA Rebuild Charter

Purpose

GDA is moving from iterative salvage to a structured rebuild. Session history shows that data and some workflows often remain live, but the operator experience repeatedly regresses through brittle deploy paths, proxy/auth drift, workflow mismatches, and frontend instability.[cite:215][cite:218][cite:251]

The rebuild will preserve the canonical data core and rebuild the operator-facing product around a simpler, more durable architecture. The end state is a trustworthy operating system for business development and capture work, not a loose collection of partially overlapping tools.[cite:215][cite:251]

Strategic decision

The rebuild will use React as the primary production UI, Postgres as the system of record, and n8n as the workflow engine.[cite:215] React-facing endpoints must use a standard JSON response envelope so the frontend can distinguish healthy data, empty data, auth failures, and workflow failures without guesswork.[cite:216]

The current environment will be treated as untrusted until re-verified. Existing workflows may be reused only if they meet the new contract, testing, and reliability rules.[cite:216][cite:218][cite:251]

Product scope

The product will be rebuilt around five primary tools:

- Ops Tracker
- Financial Bible
- Prompt Architect
- Compliance Matrix
- Proposal Review

Launchpad will be the front door to the platform. Morning Brief and SitRep are removed from active scope, and duplicative document/intel/chat/research surfaces will be consolidated into one future workbench instead of remaining as separate tools.[cite:215][cite:251]

Product rules

A persistent, financial-only KPI header will appear on every page. It will contain Orders, Sales, EBIT, ROS, Funded Backlog, Backlog, and Gross Profit, and each KPI will click through to the exact detail view it represents.[cite:251]

Below the persistent KPI strip, every page will have a local summary row that is specific to the active tool. On Launchpad, that row will summarize the platform with items such as top scoring opportunities, risks, decisions, due-soon items, and fast-track signals.[cite:198][cite:201]

An opportunity is not pipeline until it is qualified. This distinction will be enforced in the product model, UI, and reporting logic so the platform stops mixing the discovery universe with the qualified portfolio.[cite:198][cite:201]

Launchpad first

The first successful release is a trustworthy Launchpad. It should show the top 10 scoring opportunities, a summary of decisions coming up, active risks, due-soon items, relevant business news placeholders, and entry points into the five core tools.[cite:198][cite:201]

The first implementation should avoid charts and graphs. Tables, counts, lists, and clear navigation take priority; charts can return later after the underlying data logic is proven correct.[cite:218]

Architecture decisions

The rebuild will use four clean layers:

1. Data layer: Postgres is the source of truth for opportunities, fast-track signals, and business state.[cite:198][cite:215]
2. API layer: React only calls safe relative `/api/...` routes, with standardized JSON responses.[cite:215][cite:216]
3. Frontend layer: a new simplified React shell built around Launchpad and the five primary tools.[cite:215]
4. Release and QA layer: smoke tests, API checks, business-journey E2E, and visible failure diagnostics.[cite:218]

Lockdown policy

When a feature works, it should be locked down with product controls rather than trusted informally. That means source control as the truth, standardized endpoint contracts, explicit test coverage, a single deploy path, archived duplicate surfaces, and visible QA checks before and after release.[cite:215][cite:216][cite:218]

A feature will not be considered done because it rendered once. It is done only when it survives deploy, passes the agreed tests, and fails visibly and diagnosably when something upstream breaks.[cite:216][cite:218]

Success criteria

The rebuild is successful when Launchpad is trustworthy, opportunities and pipeline are clearly separated, the persistent financial KPI header works across all pages, and core user journeys pass repeatable tests without mystery failures.[cite:215][cite:218]